Compilers Theory
3CS – CS309-Credit

Introduction to Compilers

Dr. Lamiaa Hassaan

What is a Compiler?

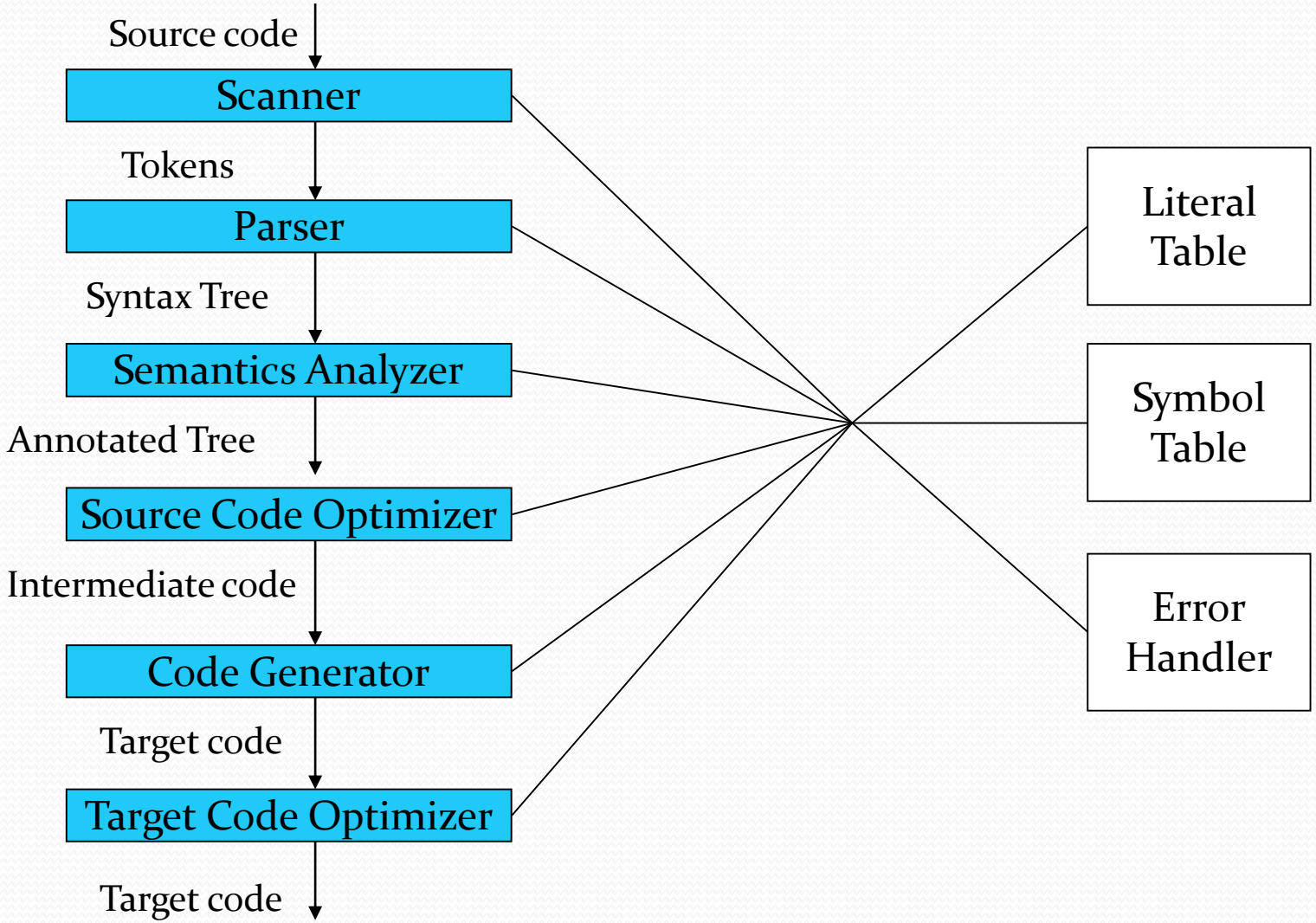
- A compiler translates (or compiles) a program written in a high-level programming language that is suitable for human programmers into the low-level machine language that is required by computers. During this process, the compiler will also attempt to spot and report obvious programmer mistakes.

- An interpreter is another way of implementing a programming language. Interpretation shares many aspects with compiling.

Compilers vs. Interpreters

Interpreter	Compiler
Translates program one statement at a time.	Scans the entire program and translates it as a whole into machine code.
Interpreters usually take less amount of time to analyze the source code. However, the overall execution time is comparatively slower than compilers.	Compilers usually take a large amount of time to analyze the source code. However, the overall execution time is comparatively faster than interpreters.
No intermediate object code is generated, hence are memory efficient.	Generates intermediate object code which further requires linking, hence requires more memory.
Programming languages like JavaScript, Python, Ruby use interpreters.	Programming languages like C, C++, Java use compilers.

Phases of a Compiler



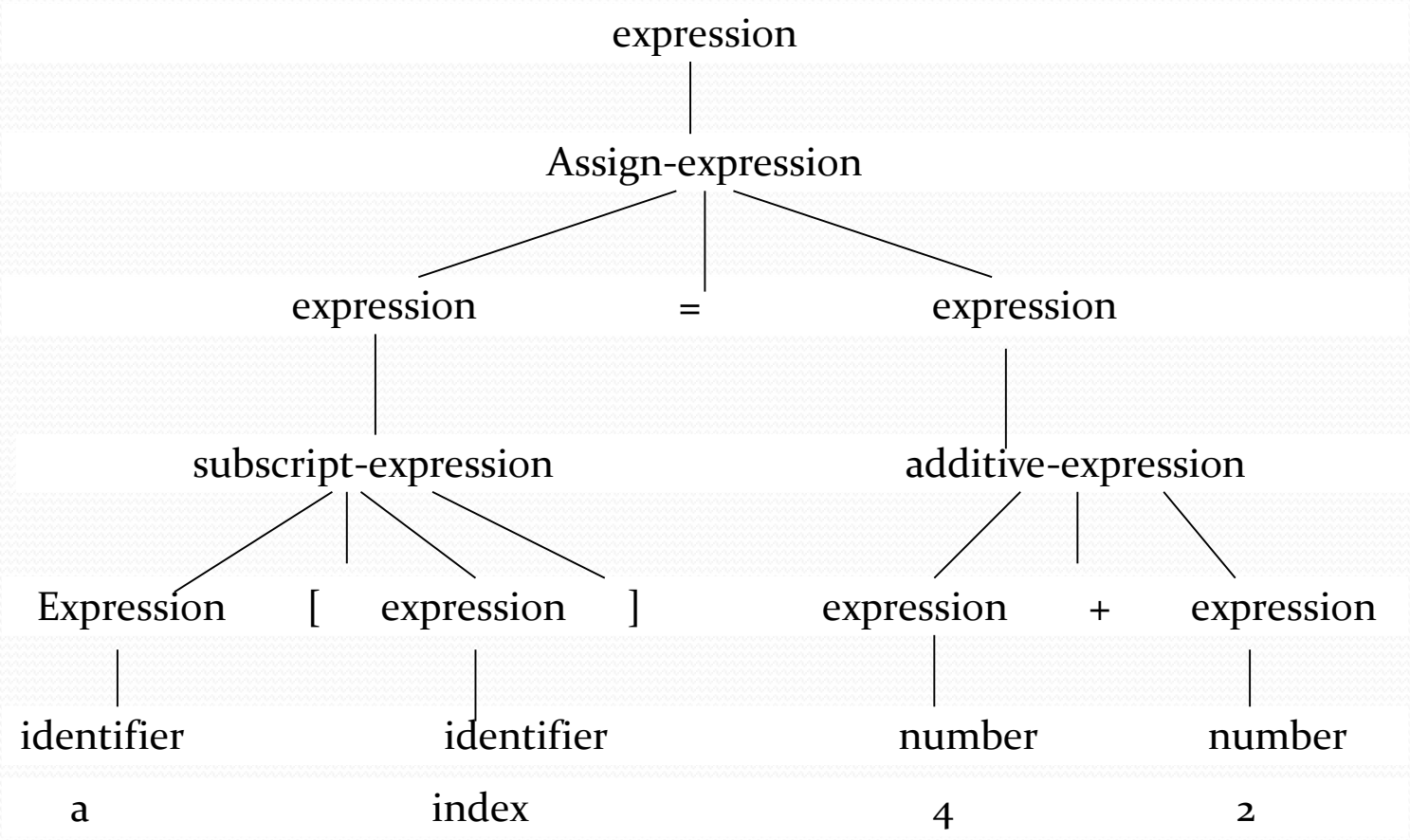
1- Lexical Analysis

- it collects sequences of characters into meaningful units called tokens
- An example: a[index]=4+2
  - a identifier
  - [left bracket
  - index identifier
  -] right bracket
  - = assignment
  - 4 number
  - + plus sign
  - 2 number

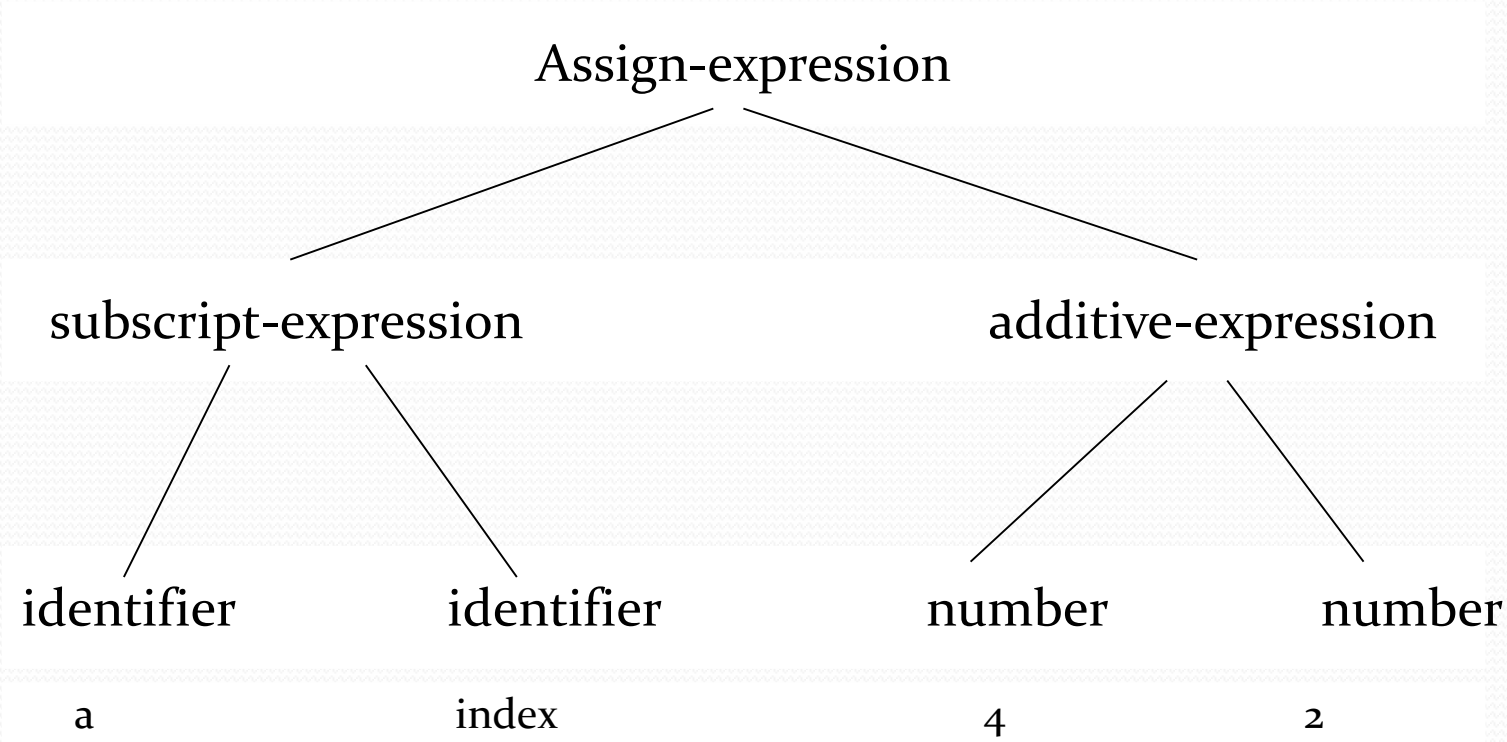
2- Syntax Analysis

- it determines the structure of the program
- The results of syntax analysis are a parse tree or a syntax tree
- An example: a[index]=4+2
  - Parse tree
  - Syntax tree (abstract syntax tree)

The Parse Tree



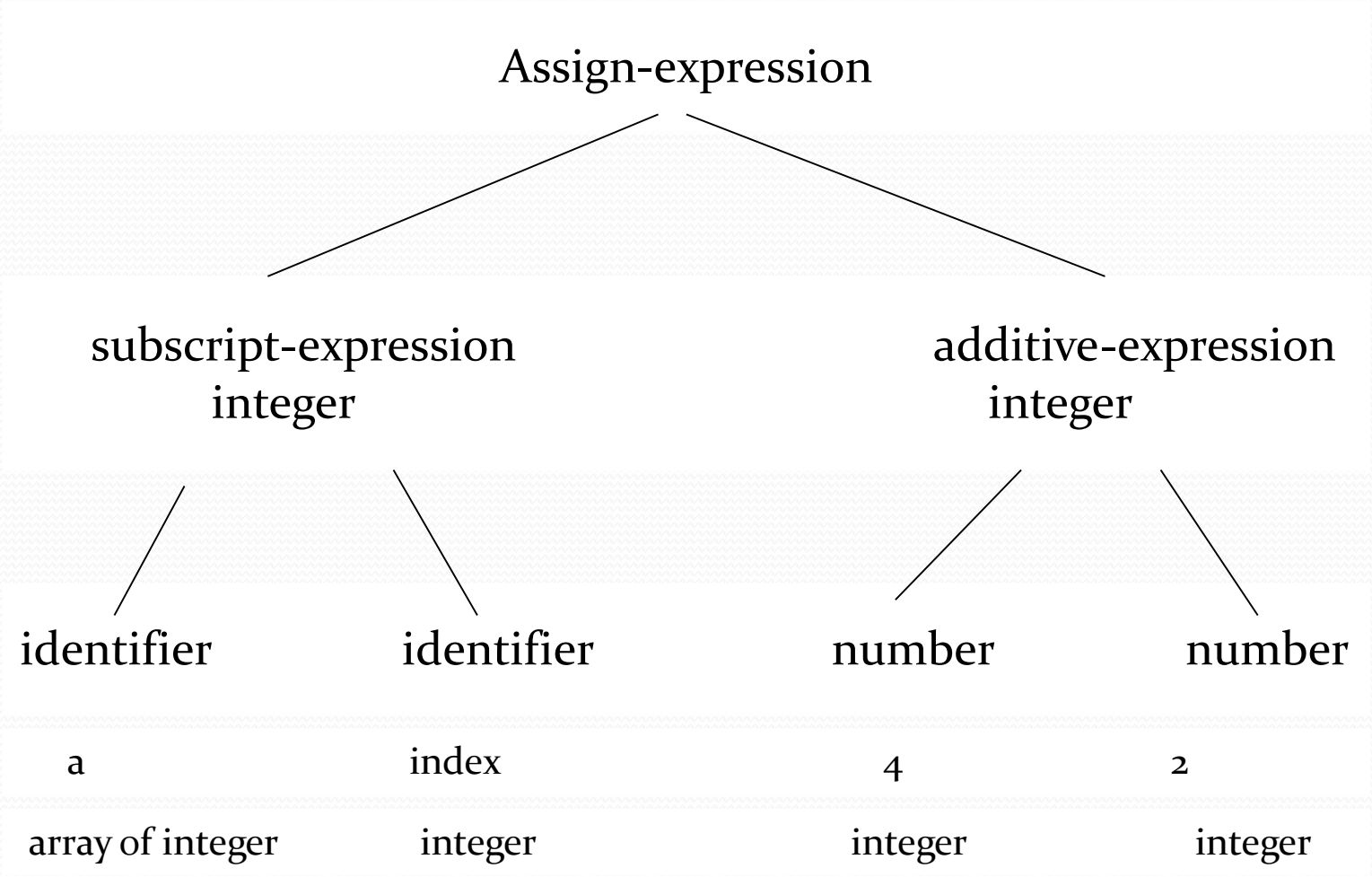
The Syntax Tree



3- The Semantic Analyzer

- The semantics of a program are its "meaning", as opposed to its syntax, or structure, that
  - determines some of its running time behaviors prior to execution.
- Static semantics: declarations and type checking
- Attributes: The extra pieces of information computed by semantic analyzer
- An example: a[index]=4+2
  - The syntax tree annotated with attributes

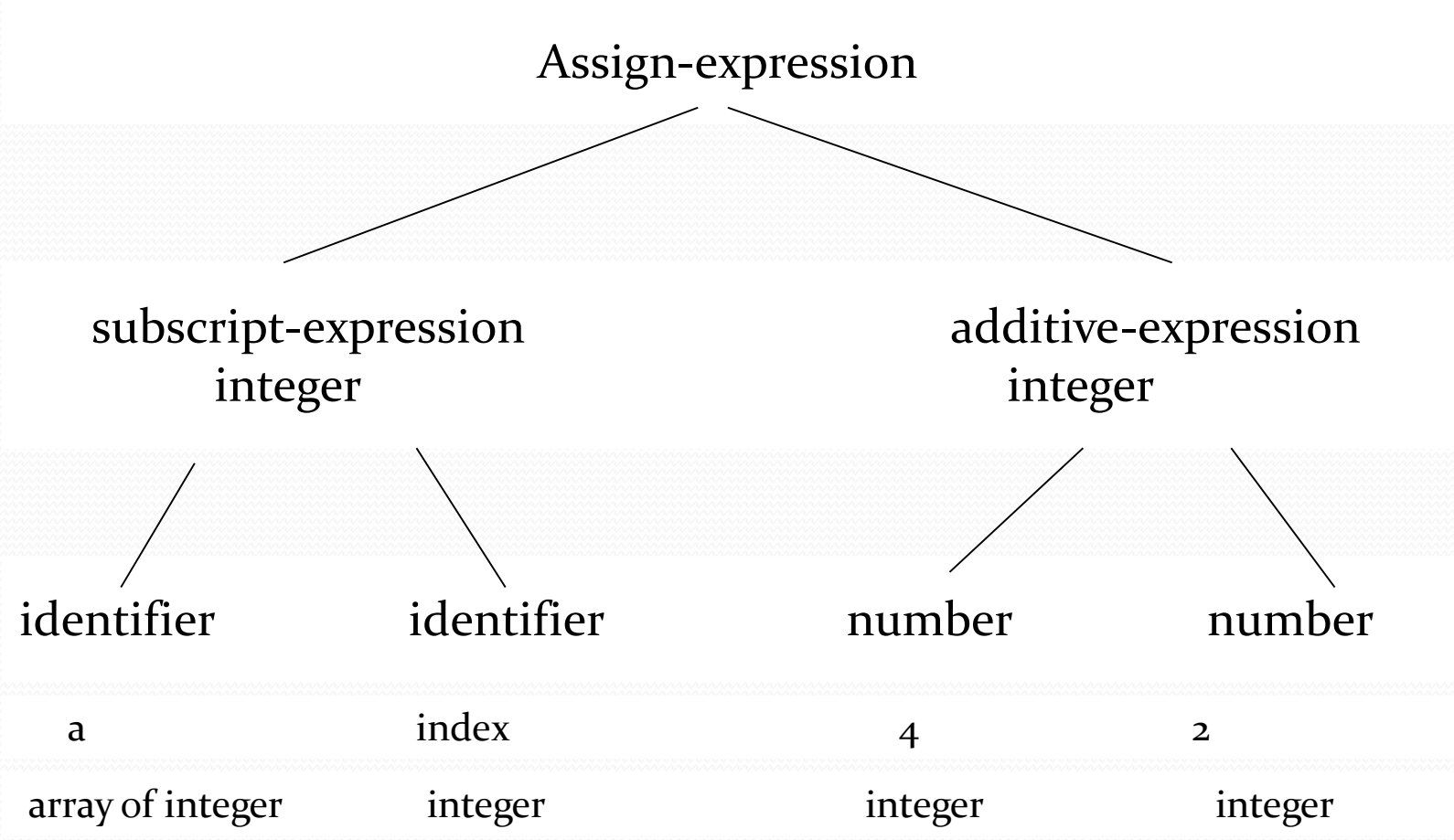
The Annotated Syntax Tree



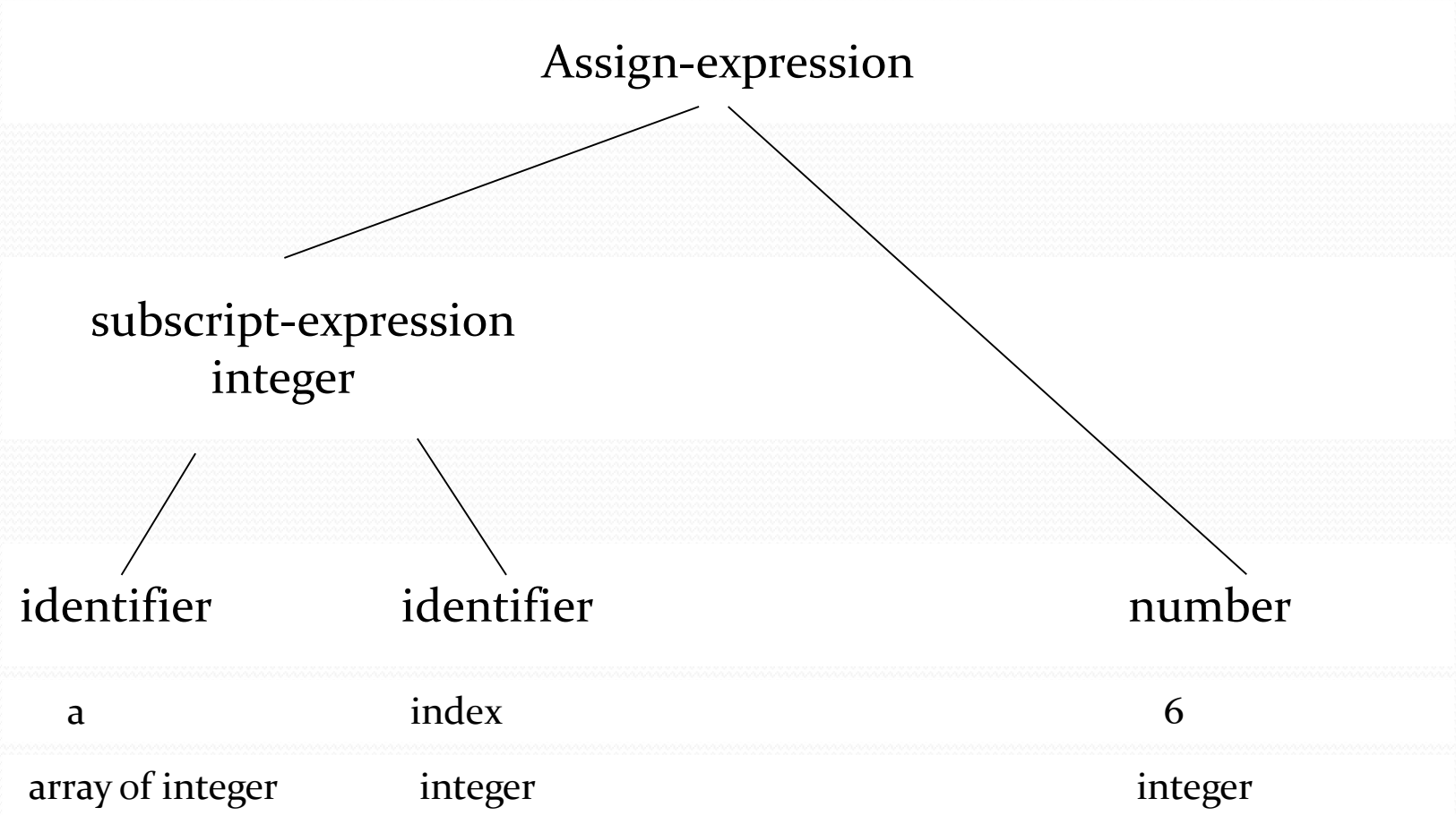
4- The Source Code Optimizer

- The earliest point of most optimization steps is just after semantic analysis
- The code improvement depends only on the source code, and as a separate phase
- Individual compilers exhibit a wide variation in optimization kinds as well as placement
- An example: a[index]=4+2
  - Constant folding performed directly on annotated tree
  - Using intermediate code: three-address code, p-code

Optimizations on Annotated Tree



Optimizations on Annotated Tree



Optimization on Intermediate Code

```
t = 4 + 2
a[index]=t
```

↓

```
t = 6
a[index]=t
```

↓

```
a[index]=6
```

5- The Code Generation

- It takes the intermediate code or IR and generates code for target machine
- The properties of the target machine become the major factor:
  - Using instructions and representation of data
- An example: a[index]=4+2
  - Code sequence in a hypothetical assembly language

A possible code sequence

a[index]=6

```
MOV R0, index
MUL R0,2
MOV R1,&a
ADD R1,R0
MOV *R1,6
```

6- The Target Code Optimizer

- It improves the target code generated by the code generator:
  - Address modes choosing
  - Instructions replacing
  - As well as redundant eliminating

```
MOV R0, index
MUL R0,2
MOV R1,&a
ADD R1,R0
MOV *R1,6
```

→

```
MOV R0, index
SHL R0
MOV &a[R1],6
```

Principle Data Structure for Communication among Phases

- TOKENS
  - A scanner collects characters into a token, as a value of an enumerated data type for tokens
  - May also preserve the string of characters or other derived information, such as name of identifier, value of a number token
  - A single global variable or an array of tokens
- THE SYNTAX TREE
  - A standard pointer-based structure generated by parser
  - Each node represents information collect by parser or later, which maybe dynamically allocated or stored in symbol table
  - The node requires different attributes depending on kind of language structure, which may be represented as variable record.

Principle Data Structure for Communication among Phases

- THE SYMBOL TABLE
  - Keeps information associated with identifiers: function, variable, constants, and data types
  - Interacts with almost every phase of compiler.
  - Access operation need to be constant-time
  - One or several hash tables are often used,
- THE LITERAL TABLE
  - Stores constants and strings, reducing size of program
  - Quick insertion and lookup are essential

Principle Data Structure for Communication among Phases

- INTERMEDIATE CODE
  - Kept as an array of text string, a temporary text, or a linked list of structures, depending on kind of intermediate code (e.g. three-address code and p-code)
  - Should be easy for reorganization
- TEMPORARY FILES
  - Holds the product of intermediate steps during compiling
  - Solve the problem of memory constraints or back-patch addressed during code generation